Insper

MEGADADOS

Introdução ao Apache Spark

Processamento distribuído com RDDs

Marcel Parolin Jackowski mjack@insper.edu.br
Maciel Vidal macielcv@insper.edu.br

Quando os dados não cabem em uma máquina

Scale up

Uma máquina maior: mais memória, mais disco, mais CPU.

Fica cada vez mais cara e esbarra em um limite físico.

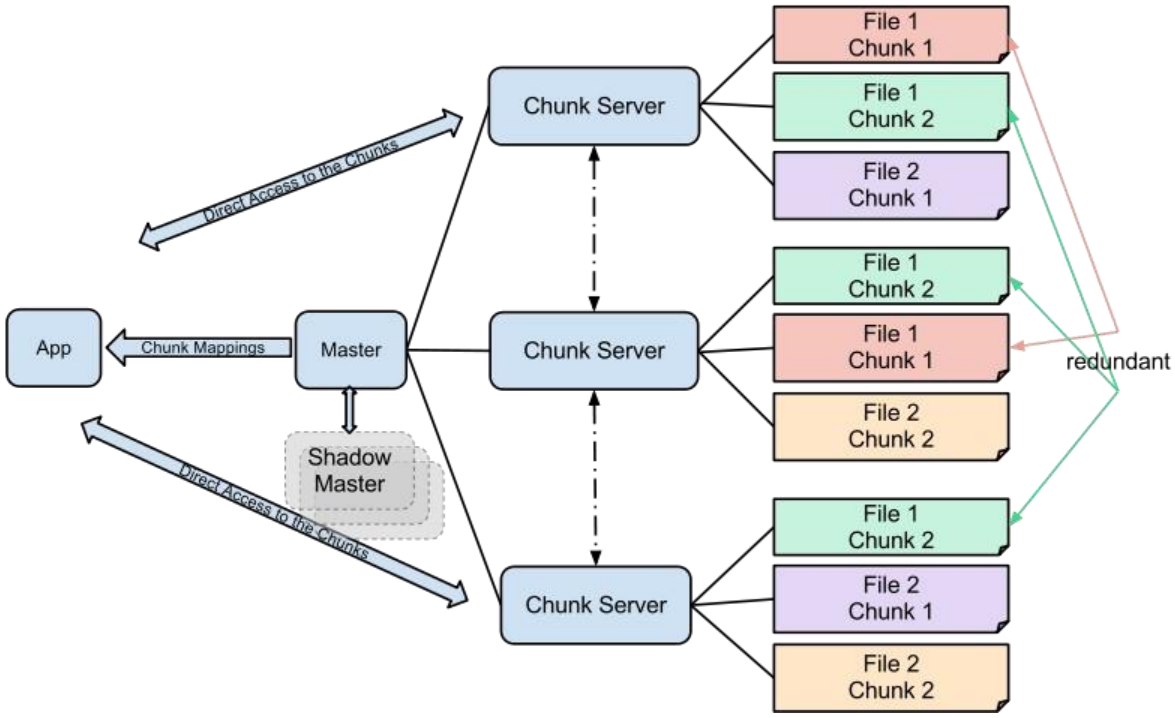
Scale out

Muitas máquinas comuns trabalhando juntas em um cluster.

Exige distribuir os dados e lidar com falhas de máquinas.

Em um cluster grande, falhas deixam de ser exceção e viram rotina. O software precisa tolerá-las.

Google File System (2003)



Ghemawat, Gobioff e Leung. “The Google File System”. SOSP 2003.

Chunks: arquivos são divididos em blocos grandes (64 MB no artigo) espalhados pelo cluster.

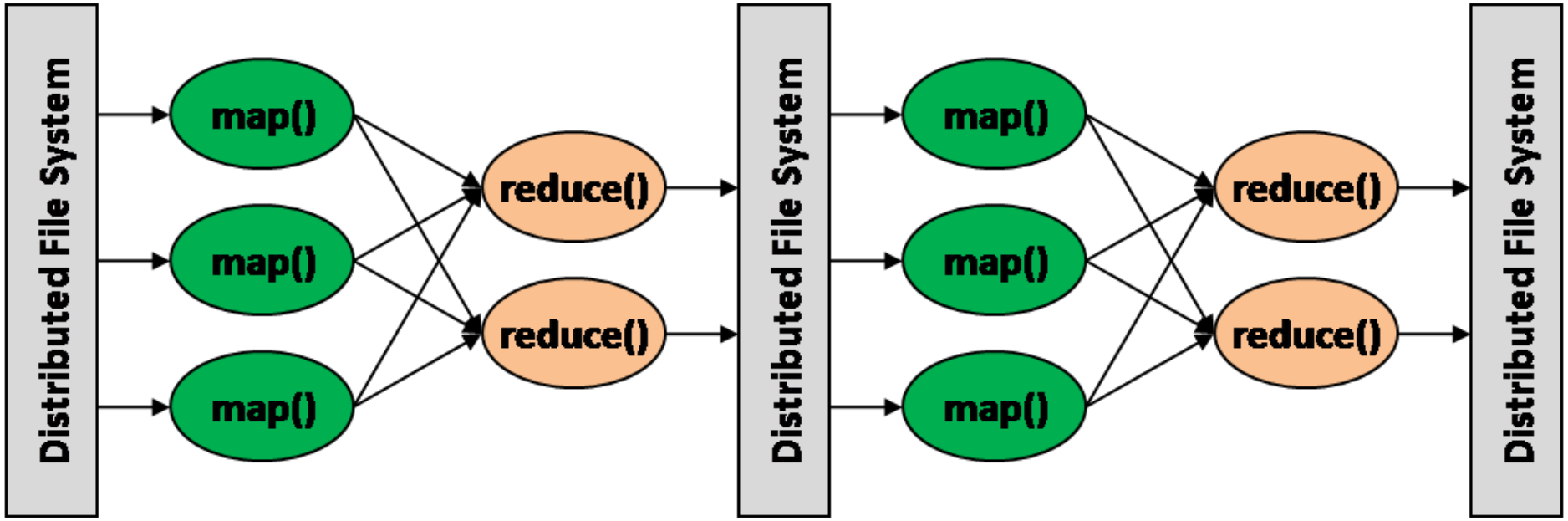
Réplicas: cada chunk é copiado em várias máquinas (3 por padrão).

Master: guarda só os metadados, ou seja, onde está cada chunk.

Acesso direto: a aplicação lê os dados dos chunk servers, sem passar pelo master.

Insper

MapReduce (2004)



Os **maps** processam partes dos dados em paralelo. Os **reduces** agregam os resultados por chave. A saída de cada etapa é gravada no sistema de arquivos distribuído.

Dean e Ghemawat. “MapReduce: Simplified Data Processing on Large Clusters”. OSDI 2004.

Insper

A IDEIA CENTRAL DO MAPREDUCE

Levar a computação até os dados, e não o contrário !

Muito mais eficiente quando a massa de dados é enorme: mover código custa kilobytes, mover dados custa terabytes.

MapReduce na prática: contar palavras

```
map(nome, documento):
   para cada palavra w em documento:
      emite (w, 1)

reduce(palavra, contagens):
   soma = 0
   para cada c em contagens:
      soma += c
   emite (palavra, soma)
```

Entrada

```
to be or not to be
```

Depois do map

```
(to,1) (be,1) (or,1)
(not,1) (to,1) (be,1)
```

Agrupado por chave (shuffle)

```
to:[1,1] be:[1,1]
or:[1] not:[1]
```

Depois do reduce

```
(to,2) (be,2) (or,1) (not,1)
```

O que faltava ao MapReduce

Disco entre etapas

Cada job lê do sistema de arquivos distribuído e grava o resultado de volta nele.

Algoritmos iterativos

Aprendizado de máquina e grafos repetem passos sobre os mesmos dados, relendo-os do disco a cada volta.

Programação trabalhosa

Encadear várias etapas map e reduce exige muito código de baixo nível.

O Spark responde com uma ideia simples: manter os dados em memória sempre que possível, sem gravar resultados intermediários em disco.

Apache Spark

O que é

Um framework para processamento distribuído

Onde roda

Sobre a Java Virtual Machine

Implementação

Escrito em Scala

Linguagens

Java, Scala e Python (e também R e SQL)

Baseado no paradigma funcional: um programa é uma composição de funções como map e filter.

Demonstração em Scala

```scala
// 1. Definição de uma estrutura de dados imutável (Case Class)
case class Produto(nome: String, preco: Double)

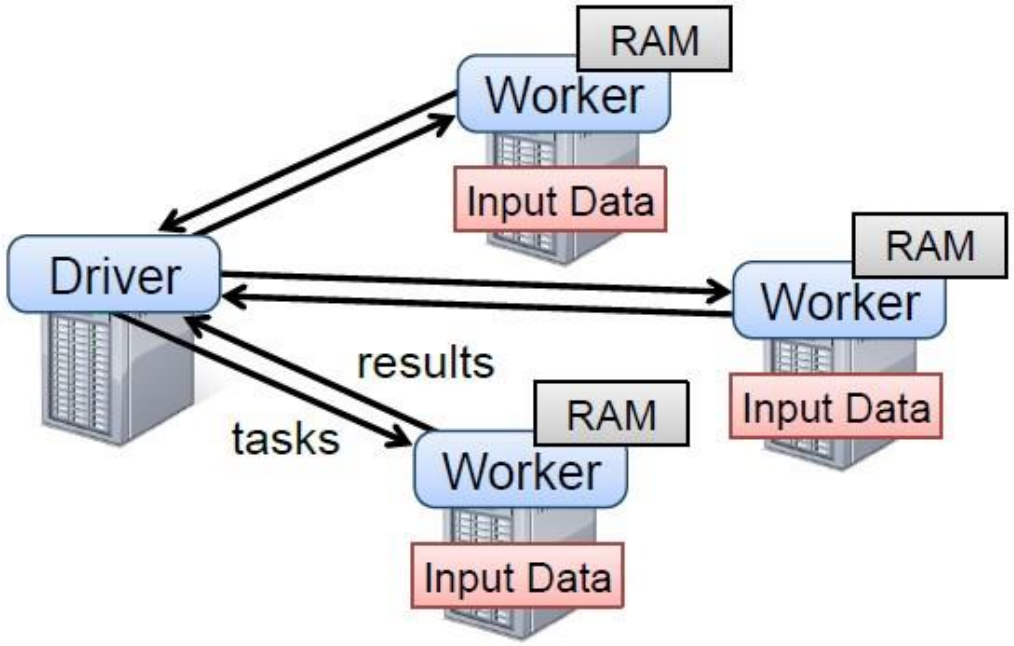
object ExemploScala:
  def main(args: Array[String]): Unit =

    // 2. Criação de uma lista de produtos
    val produtos = List(
      Produto("Notebook", 4500.0),
      Produto("Mouse", 150.0),
      Produto("Teclado", 300.0),
      Produto("Monitor", 1200.0)
    )

    // 3. Paradigma Funcional: Filtrando e transformando os dados
    // Seleciona produtos acima de 200 reais e extrai apenas os nomes
    val produtosCaros = produtos
      .filter(p ⇒ p.preco > 200.0)
      .map(p ⇒ p.nome)

    // 4. Exibindo o resultado
    println(s"Produtos acima de R$$200: $produtosCaros")
```

Arquitetura: um driver e vários workers



Driver: executa o seu programa e divide o trabalho em tasks.

Workers: executam as tasks sobre a parte dos dados que guardam, na memória (RAM).

Resultados: voltam ao driver, que os junta.

Existe somente um driver por aplicação, se ele cair a aplicação inteira cai.

Zaharia et al. "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing". NSDI 2012. spark.apache.org

Insper

O conceito fundamental: o RDD

Resilient Distributed Dataset: uma coleção de dados que o Spark espalha pelo cluster e sabe recalcular.

Distributed

O dataset é dividido em partições espalhadas por várias máquinas.

Resilient

Se uma máquina cai, as partições que estavam nela são realocadas e recalculadas em outras máquinas.

Imutável

Um RDD não muda. Cada transformação gera um novo RDD, e esse histórico permite recalcular o que for perdido.

Transformations e actions

Transformations

Atuam sobre um RDD e **retornam um novo RDD**.

Não calculam nada na hora: o Spark só registra a operação. Isso é a *lazy evaluation*.

map, flatMap, filter, reduceByKey

Actions

Atuam sobre um RDD e **retornam um valor concreto** ou produzem um efeito final, como gravar em disco.

É nelas que a computação realmente acontece.

reduce, count, take, collect, saveAsTextFile

Encadeamos transformations; uma action dispara tudo de uma vez.

As operações mais usadas

Operação	Tipo	O que faz
map	transformation	transforma cada elemento (1 para 1)
flatMap	transformation	um elemento vira vários (1 para N)
filter	transformation	mantém só o que passa no teste
reduceByKey	transformation	agrupa por chave e combina os valores
reduce	action	combina todos os elementos em um valor
count, take, collect	action	devolvem valores ao driver

> **Atenção:** reduce é uma action e devolve um valor. reduceByKey é uma transformation e devolve um RDD.

Contagem de palavras em PySpark

```python
from pyspark import SparkContext
sc = SparkContext("local[*]", "WordCount")

# transformations: nada é calculado ainda
linhas = sc.textFile("livro.txt")
palavras = linhas.flatMap(lambda l: l.split())
pares = palavras.map(lambda w: (w, 1))
contagem = pares.reduceByKey(lambda a, b: a + b)

# action: aqui o Spark executa tudo
print(contagem.take(10))
```

Lazy evaluation passo a passo

1 · Definir

Você encadeia transformations. O Spark apenas anota o plano de execução.

2 · Disparar

Você chama uma action, como take ou count. Só agora o trabalho é necessário.

3 · Executar

O driver envia tasks aos workers, que processam as partições e devolvem o resultado.

O Spark enxerga a cadeia inteira antes de executar. Calcula só o que a action precisa e, se uma partição se perder, sabe recalculá-la.

Além dos RDDs: o ecossistema Spark

Spark SQL e DataFrames

Tabelas distribuídas e consultas SQL. É a API recomendada na maior parte dos casos.

MLlib

Algoritmos de aprendizado de máquina que rodam de forma distribuída.

Structured Streaming

Processamento de dados que chegam continuamente, em fluxo.

GraphX

Computação sobre grafos, como redes sociais e links entre páginas.

Todos rodam sobre o mesmo núcleo, o Spark Core, que implementa RDDs, tasks e tolerância a falhas.

Quando usar Spark

Situação	Ferramenta
Dados que cabem em uma só máquina	DuckDB, Polars
Streaming com latência de milissegundos	Apache Flink
Consultas SQL interativas, muitos usuários	Trino
ETL em lote grande, ML, terabytes	Apache Spark

> Regra prática: comece pela ferramenta mais simples que resolve. Um cluster só compensa quando os dados não cabem em uma máquina.

Exercício sugerido

Abra o guia de programação de RDDs: spark.apache.org/docs/latest/rdd-programming-guide.html

1. Rode a contagem de palavras em um texto à sua escolha.
2. Mostre as 10 palavras mais frequentes (dica: sortBy).
3. Descarte palavras muito comuns usando filter.
4. Troque take(10) por collect(). Quando isso é perigoso?

Referências

GFS. Ghemawat, S.; Gobioff, H.; Leung, S.-T. The Google File System. 19th ACM Symposium on Operating Systems Principles, Lake George, NY, 2003.

MapReduce. Dean, J.; Ghemawat, S. MapReduce: Simplified Data Processing on Large Clusters. OSDI 2004, San Francisco, CA.

Spark. Zaharia, M. et al. Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. NSDI 2012.

Documentação. spark.apache.org e spark.apache.org/docs/latest/rdd-programming-guide.html

Figuras. Diagrama do MapReduce: dzone.com/articles/how-hadoop-mapreduce-works

Insper

Insper

www.insper.edu.br